

FAKE JOB POSTING DETECTION USING MACHINE LEARNING

Project Report

Submitted By

Sachin Tripathi

B.Tech Computer Science & Engineering (Artificial Intelligence & Machine
Learning)

3rd Year (5th Semester)

Swami Devi Dayal Institute of Engineering & Technology

Submitted To

3Skills AI & ML Internship

August 2026

Table of Contents

Abstract	4
Chapter 1: Introduction	5
Chapter 2: Problem Statement	6
Chapter 3: Objectives	7
Chapter 4: Dataset Description	8
Chapter 5: Exploratory Data Analysis (EDA)	9
5.1 Missing Value Analysis	9
5.2 Fraudulent Class Distribution	10
5.3 Duplicate Record Analysis	11
Chapter 6: Data Preprocessing.....	12
6.1 Handling Missing Values	12
6.2 Combining Important Text Features	12
6.3 Text Cleaning.....	13
6.4 Stopword Removal	13
6.5 Stemming	14
Chapter 7: Feature Engineering	15
Chapter 8: Feature Combination Experiments	16
Chapter 9: Machine Learning Models.....	17
9.1 Logistic Regression	17
9.2 Multinomial Naive Bayes	18
9.3 Random Forest Classifier	18
9.4 Support Vector Machine (Linear SVM).....	19
Chapter 10: Hyperparameter Tuning	21
Chapter 11: Model Evaluation.....	22
Chapter 12: Model Comparison.....	23
Chapter 13: Prediction System	24
Chapter 14: Results and Discussion.....	25
Chapter 15: Conclusion.....	26

Chapter 16: Future Scope	27
Chapter 17: References	28

Abstract

Online job portals have become one of the primary platforms for recruitment. Along with the increasing number of genuine job opportunities, fraudulent job postings have also increased significantly. These fake job advertisements often mislead job seekers, collect personal information, or demand money in the name of registration or processing fees. Identifying such fraudulent postings manually is difficult because of the large volume of advertisements published every day.

The objective of this project is to develop a Machine Learning based Fake Job Posting Detection System capable of classifying job advertisements as either genuine or fraudulent. The project focuses mainly on Natural Language Processing (NLP) techniques for analyzing textual information such as job title, company profile, job description, requirements, and benefits.

The dataset was preprocessed by handling missing values, cleaning textual data, removing stop words, and applying stemming. The cleaned text was transformed into numerical features using the TF-IDF Vectorizer. Multiple machine learning algorithms including Logistic Regression, Naive Bayes, Random Forest, and Support Vector Machine were trained and evaluated. Hyperparameter tuning was performed using GridSearchCV to improve the performance of the Support Vector Machine.

Experimental results showed that the Support Vector Machine achieved the best overall performance and was selected as the final model. A prediction system was also developed to classify unseen job postings as genuine or fraudulent.

Chapter 1: Introduction

The recruitment process has changed significantly with the growth of online job portals. Thousands of companies publish job advertisements every day through various online platforms, allowing job seekers to apply easily from anywhere. Although these platforms provide many employment opportunities, they have also become targets for fraudulent activities.

Fake job postings are created by scammers to collect sensitive personal information, conduct financial fraud, or exploit job seekers. Since many fake advertisements appear similar to genuine ones, manually identifying fraudulent postings is both difficult and time-consuming.

Machine Learning and Natural Language Processing provide effective techniques for automatically identifying fake job postings by analyzing the textual content of advertisements. These techniques can learn patterns from historical data and classify new job postings with high accuracy.

This project develops a machine learning based system that automatically predicts whether a job posting is genuine or fraudulent using textual information extracted from the dataset.

Chapter 2: Problem Statement

Online recruitment platforms contain both genuine and fraudulent job advertisements. Fake postings often resemble legitimate opportunities, making it difficult for users to distinguish between them.

The primary problem addressed in this project is to build an intelligent system capable of automatically identifying fake job postings using machine learning and natural language processing techniques.

The system should analyze job-related textual information and classify each advertisement into one of the following categories:

- Genuine Job Posting
- Fraudulent Job Posting

Such a system can help reduce recruitment fraud and improve trust in online recruitment platforms.

Chapter 3: Objectives

The major objectives of this project are:

- To study the characteristics of genuine and fraudulent job postings.
- To perform exploratory data analysis on the dataset.
- To clean and preprocess textual information.
- To convert textual information into numerical form using TF-IDF Vectorization.
- To train multiple machine learning classification algorithms.
- To compare the performance of different models.
- To perform hyperparameter tuning for improving model performance.
- To develop a prediction system capable of classifying unseen job postings.
- To identify the most suitable model for fake job detection.

Chapter 4: Dataset Description

The dataset used in this project is the Fake Job Postings Dataset, which contains real-world job advertisements collected from online recruitment platforms.

The dataset contains both genuine and fraudulent job postings along with multiple structured and textual attributes.

Some important features available in the dataset include:

- Job Title
- Location
- Department
- Salary Range
- Company Profile
- Job Description
- Requirements
- Benefits
- Employment Type
- Required Experience
- Required Education
- Industry
- Function
- Fraudulent (Target Variable)

The target variable Fraudulent contains two classes:

- 0 → Genuine Job
- 1 → Fraudulent Job

The dataset consists of 17,880 job postings with 18 attributes.

Several columns contain missing values, making preprocessing an important step before training the machine learning models.

Chapter 5: Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is one of the most important stages of any machine learning project. It helps in understanding the dataset before model training and provides insights into the quality, distribution, and characteristics of the available data.

In this project, Exploratory Data Analysis was performed on the Fake Job Postings dataset to understand its structure, identify missing values, detect duplicate records, and analyze the distribution of genuine and fraudulent job postings.

The dataset contains 17,880 job postings with 18 different features, including both textual and structured attributes. The target variable is fraudulent, where:

- 0 = Genuine Job Posting
- 1 = Fraudulent Job Posting

The dataset consists of both numerical and categorical variables. Several textual columns such as title, company_profile, description, requirements, and benefits contain valuable information for fake job detection.

The following analyses were performed during the EDA stage:

- Dataset overview
- Dataset dimensions
- Data type analysis
- Missing value analysis
- Duplicate record analysis
- Fraudulent class distribution
- Feature-wise missing value distribution

Figure 1: Dataset Information (df.info() output)

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17880 entries, 0 to 17879
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   job_id                 17880 non-null  int64
1   title                  17880 non-null  object
2   location                17534 non-null  object
3   department              6333 non-null   object
4   salary_range            2868 non-null   object
5   company_profile         14572 non-null  object
6   description              17879 non-null  object
7   requirements            15184 non-null  object
8   benefits                10668 non-null  object
9   telecommuting           17880 non-null  int64
10  has_company_logo        17880 non-null  int64
11  has_questions            17880 non-null  int64
12  employment_type          14409 non-null  object
13  required_experience       10830 non-null  object
14  required_education        9775 non-null   object
15  industry                 12977 non-null  object
16  function                 11425 non-null  object
17  fraudulent               17880 non-null  int64
dtypes: int64(5), object(13)
memory usage: 2.5+ MB

```

5.1 Missing Value Analysis

Before training the machine learning models, it is important to identify missing values because incomplete information can negatively affect model performance.

The missing values of each column were calculated using the Pandas `isnull().sum()` function.

The analysis revealed that several features contain a significant number of missing values.

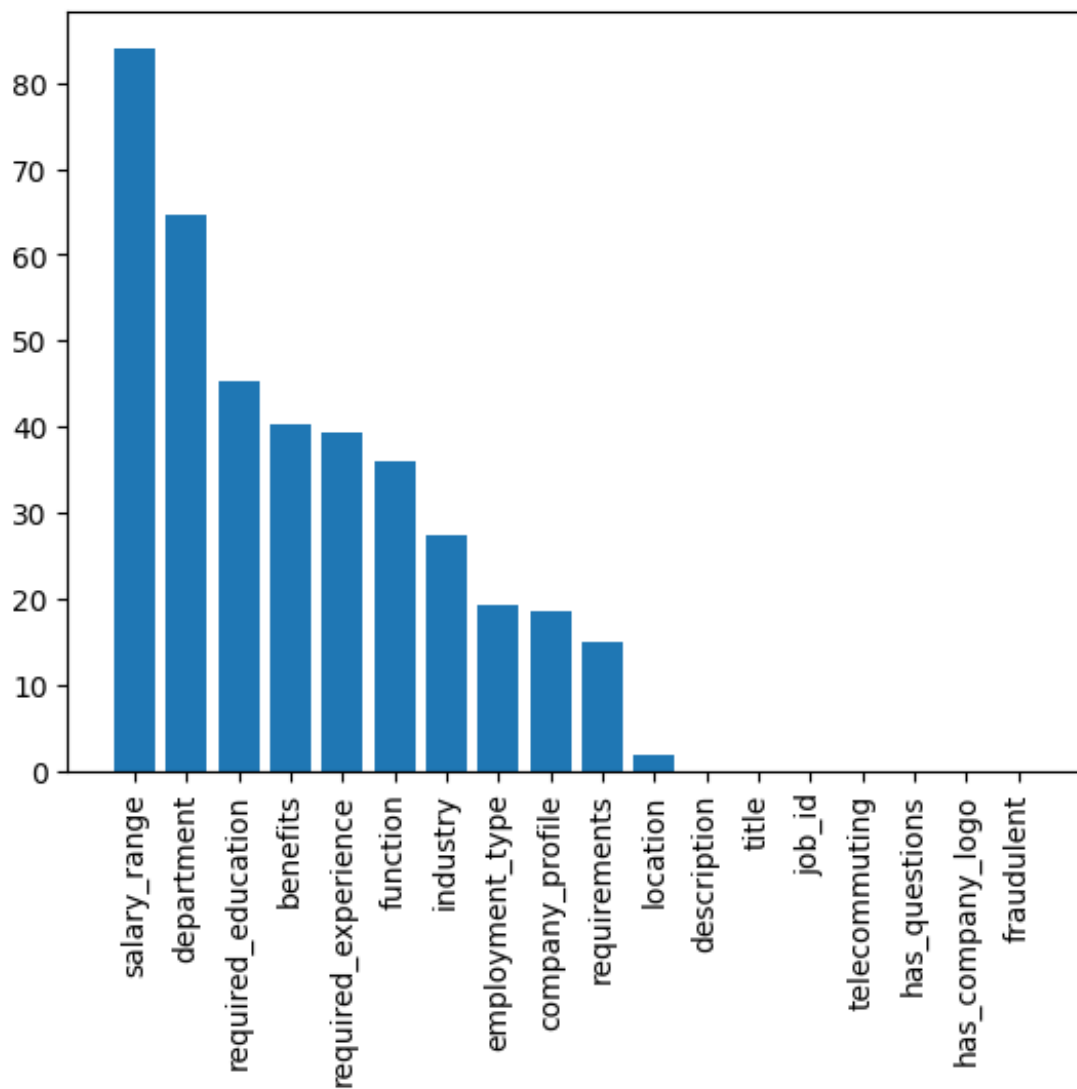
Examples include:

- Department
- Salary Range
- Company Profile
- Requirements

- Benefits
- Employment Type
- Required Experience
- Required Education
- Industry
- Function

Since these columns contain textual information, missing values were replaced with empty strings before text preprocessing.

Figure 2: Missing Value Count



5.2 Fraudulent Class Distribution

The target variable was analyzed to understand the class distribution.

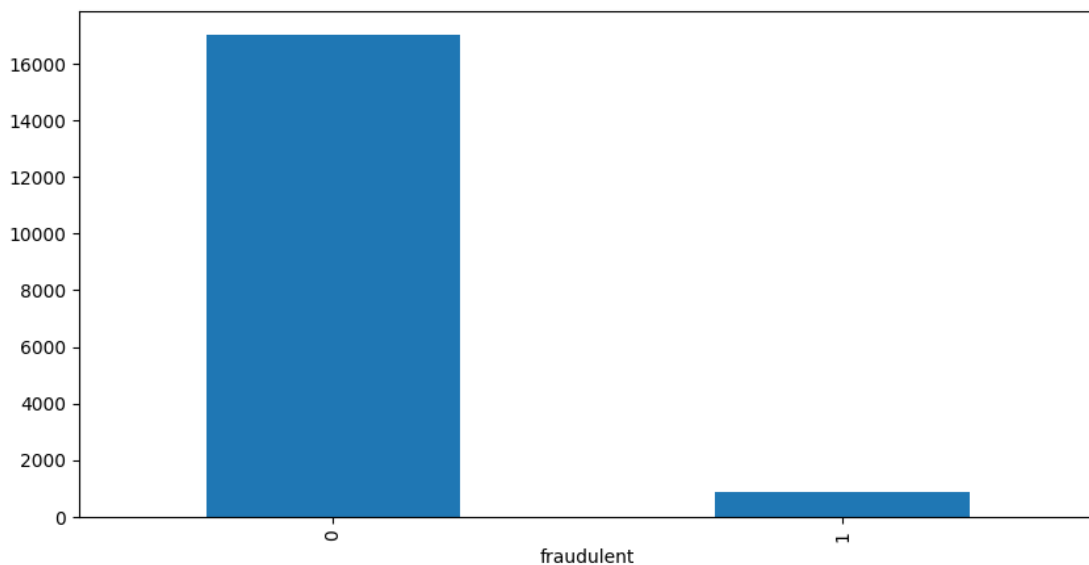
The dataset is highly imbalanced.

Approximately:

- 95% of the job postings are genuine.
- 5% of the job postings are fraudulent.

This class imbalance makes fraud detection more challenging because the model receives significantly fewer examples of fraudulent postings during training.

Figure 3: Fraudulent Class Distribution



5.3 Duplicate Record Analysis

Duplicate records can reduce model performance and introduce unnecessary redundancy.

The dataset was checked for duplicate entries.

No significant duplicate records were found that could affect the training process.

This ensured that the machine learning models learned from unique observations.

Chapter 6: Data Preprocessing

Raw textual data cannot be directly used for machine learning because algorithms work only with numerical data.

Therefore, several preprocessing steps were performed before feature extraction.

The preprocessing pipeline consisted of the following stages:

- Handling Missing Values
- Text Combination
- Lowercase Conversion
- Removing Special Characters
- Removing Numbers
- Removing Extra Spaces
- Stopword Removal
- Stemming

These preprocessing techniques improve the quality of textual information and help machine learning models learn more meaningful patterns.

6.1 Handling Missing Values

Several textual columns contained missing values.

Instead of removing those records, missing values were replaced with empty strings using the `fillna("")` function.

This prevented data loss and allowed all available information to be used during training.

6.2 Combining Important Text Features

Instead of using individual textual columns separately, multiple important columns were combined into a single text feature.

The combined columns include:

- Title
- Company Profile
- Description
- Requirements
- Benefits

Combining these features provides richer contextual information to the machine learning model and improves its ability to distinguish between genuine and fraudulent job postings.

6.3 Text Cleaning

Text cleaning is an essential step in Natural Language Processing.

The following cleaning operations were performed:

- Converted all text to lowercase.
- Removed punctuation marks.
- Removed numerical values.
- Removed extra white spaces.

These operations standardize the textual data and reduce unnecessary noise.

6.4 Stopword Removal

Stopwords are common words such as:

- the
- is
- are

- of
- and
- in

These words generally do not contribute significantly to identifying whether a job posting is genuine or fraudulent.

Therefore, English stopwords were removed using the NLTK stopwords corpus.

Removing stopwords reduces the dimensionality of the dataset and improves feature quality.

6.5 Stemming

Words often appear in different grammatical forms.

For example:

- Working
- Worked
- Works

Although these words have different spellings, they convey the same meaning.

Porter Stemmer was applied to reduce words to their root form.

Examples:

- Working → Work
- Playing → Play
- Studies → Studi

Stemming reduces vocabulary size and improves the efficiency of the machine learning models.

Chapter 7: Feature Engineering

Machine learning algorithms cannot directly process textual information.

Therefore, the cleaned text was converted into numerical vectors using the Term Frequency–Inverse Document Frequency (TF-IDF) Vectorizer.

TF-IDF assigns higher importance to words that appear frequently within a document but less frequently across all documents.

Compared to simple Count Vectorization, TF-IDF produces better numerical representations by reducing the impact of very common words.

After preprocessing, the combined text column was transformed into TF-IDF vectors, which were then used as input features for model training.

The generated feature matrix contains numerical representations of every job posting.

These vectors capture the importance of words and significantly improve classification performance.

Chapter 8: Feature Combination Experiments

To evaluate the impact of different textual attributes, multiple feature combinations were tested.

The experiments included:

Experiment 1

Only the Description column was used.

Experiment 2

The Description and Requirements columns were combined.

Experiment 3 (Final Model)

The following textual features were combined:

- Title
- Company Profile
- Description
- Requirements
- Benefits

Experimental results showed that using multiple textual features together provided richer contextual information and improved the model's ability to distinguish fraudulent job postings from genuine ones.

Based on the comparative analysis, the third feature combination was selected for building the final prediction system.

Chapter 9: Machine Learning Models

After completing data preprocessing and feature engineering, multiple machine learning classification algorithms were trained to classify job postings as Genuine or Fraudulent. Using multiple models allows a fair comparison and helps identify the most suitable algorithm for this problem.

The dataset was divided into training and testing subsets using the `train_test_split()` function. Approximately 80% of the data was used for training and 20% for testing. The same TF-IDF feature matrix was used for all models to ensure a fair comparison.

The following machine learning algorithms were implemented:

- Logistic Regression
- Multinomial Naive Bayes
- Random Forest Classifier
- Support Vector Machine (Linear SVM)

Each model was evaluated using Accuracy, Precision, Recall, F1-Score and Confusion Matrix.

9.1 Logistic Regression

Logistic Regression is a supervised machine learning algorithm widely used for binary classification problems. It estimates the probability that a given input belongs to a particular class.

In this project, Logistic Regression was trained using the TF-IDF feature vectors generated from the cleaned job posting text. The model learned patterns that differentiate genuine job advertisements from fraudulent ones.

The model produced high overall accuracy because most records in the dataset belong to the genuine class. However, its ability to detect fraudulent postings was comparatively lower than the Support Vector Machine.

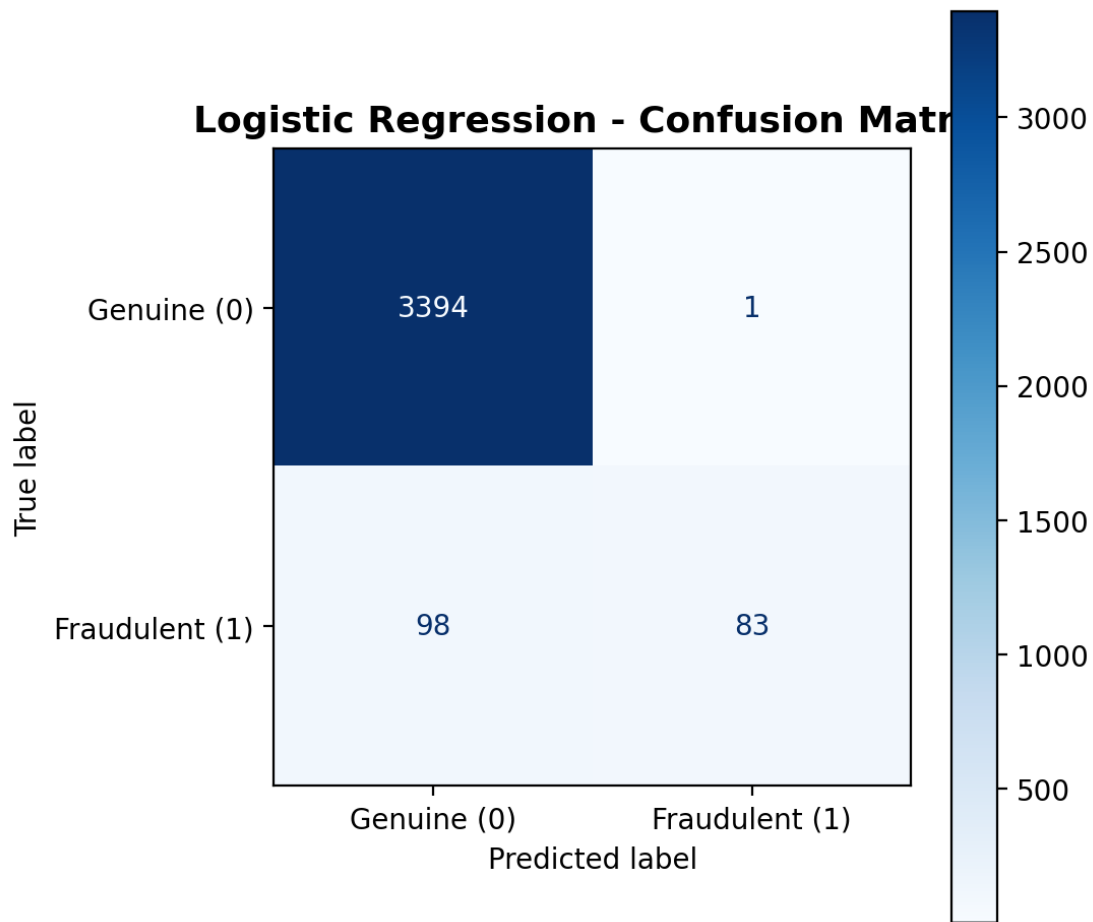
Advantages

- Simple and efficient.
- Fast training and prediction.
- Performs well for linearly separable data.

Limitations

- Performance decreases when class boundaries become more complex.
- Less effective than SVM for high-dimensional text classification.

Figure 4: Logistic Regression Confusion Matrix



9.2 Multinomial Naive Bayes

Multinomial Naive Bayes is one of the most popular algorithms for Natural Language Processing tasks. It assumes that features are conditionally independent and works efficiently with word frequency-based representations such as TF-IDF.

In this project, the algorithm was trained using the same TF-IDF feature vectors. Due to its simplicity, the model trained very quickly and produced satisfactory results.

Although Naive Bayes achieved good overall accuracy, it generated more false predictions than the Support Vector Machine, especially while identifying fraudulent job postings.

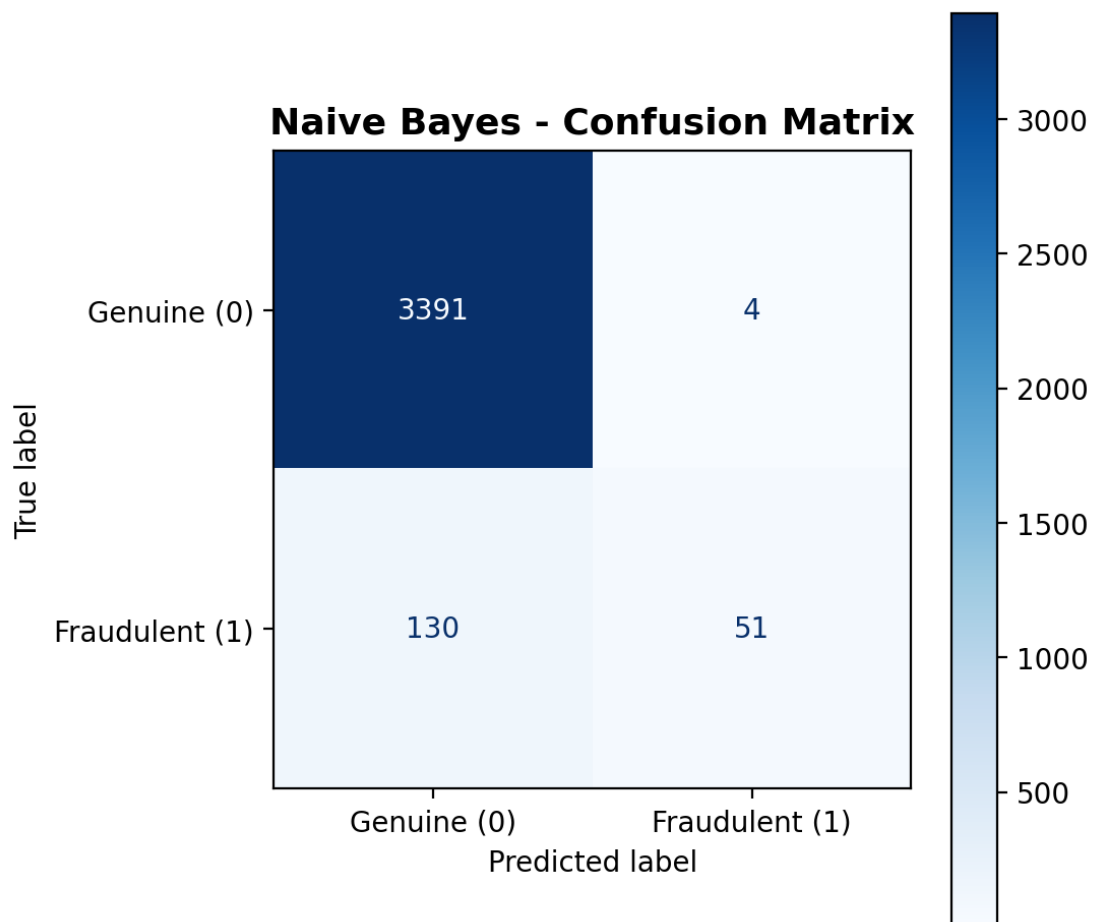
Advantages

- Very fast.
- Excellent for text classification.
- Requires less computational power.

Limitations

- Assumes feature independence.
- Performance decreases when the assumption is violated.

Figure 5: Naive Bayes Confusion Matrix



9.3 Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines the predictions of multiple decision trees.

Instead of relying on a single tree, Random Forest builds several decision trees using randomly selected subsets of the training data and combines their predictions through majority voting.

In this project, Random Forest performed better than Logistic Regression and Naive Bayes in terms of overall classification performance. However, it required more computational resources and training time compared to the other models.

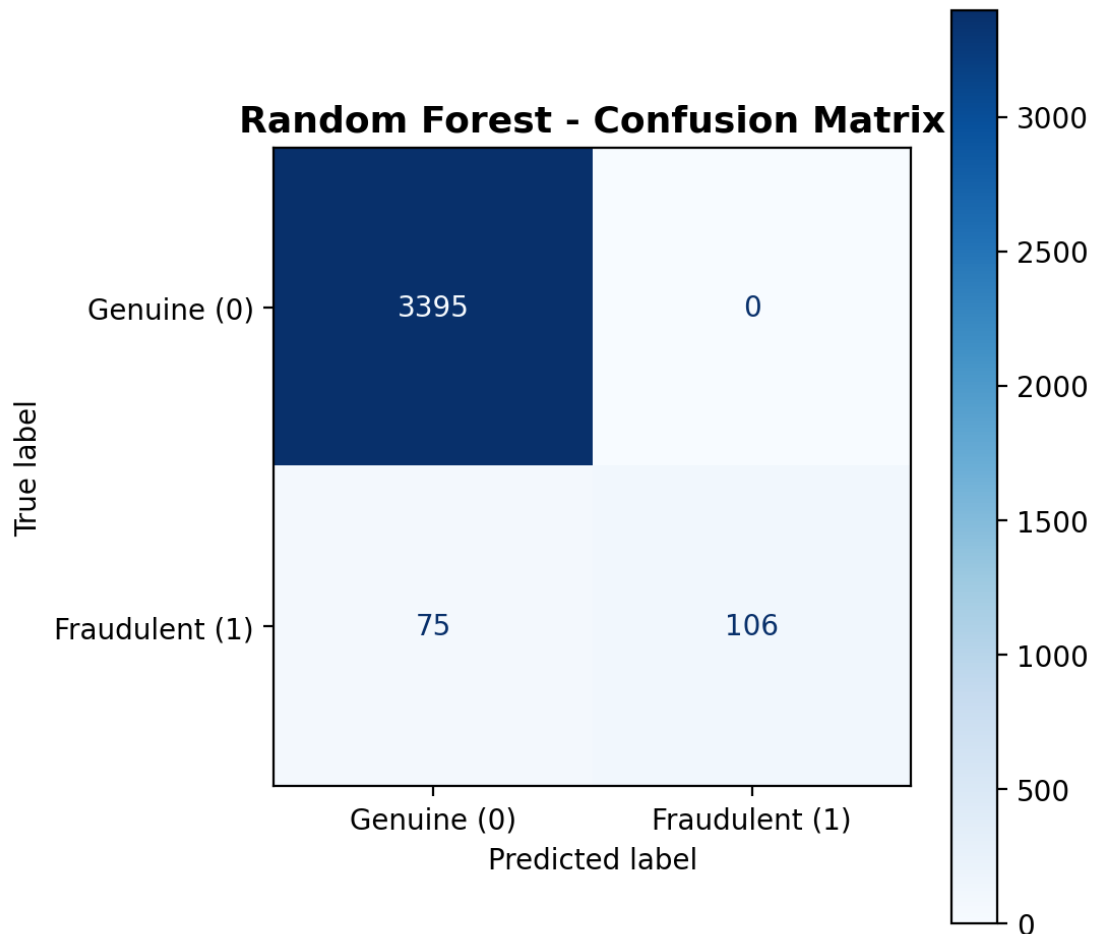
Advantages

- Handles complex relationships effectively.
- Less prone to overfitting than a single decision tree.
- Produces stable predictions.

Limitations

- Slower than simpler algorithms.
- More computationally expensive.

Figure 6: Random Forest Confusion Matrix



9.4 Support Vector Machine (Linear SVM)

Support Vector Machine (SVM) is one of the most effective algorithms for text classification problems because it performs exceptionally well on high-dimensional sparse datasets generated by TF-IDF.

The Linear Support Vector Machine was trained using the processed textual features. The algorithm identifies the optimal hyperplane that maximizes the separation between genuine and fraudulent job postings.

Among all implemented models, the Support Vector Machine achieved the best overall performance. It demonstrated higher accuracy along with better precision and recall for detecting fraudulent job advertisements.

Based on the experimental results, Support Vector Machine was selected as the final model for the prediction system.

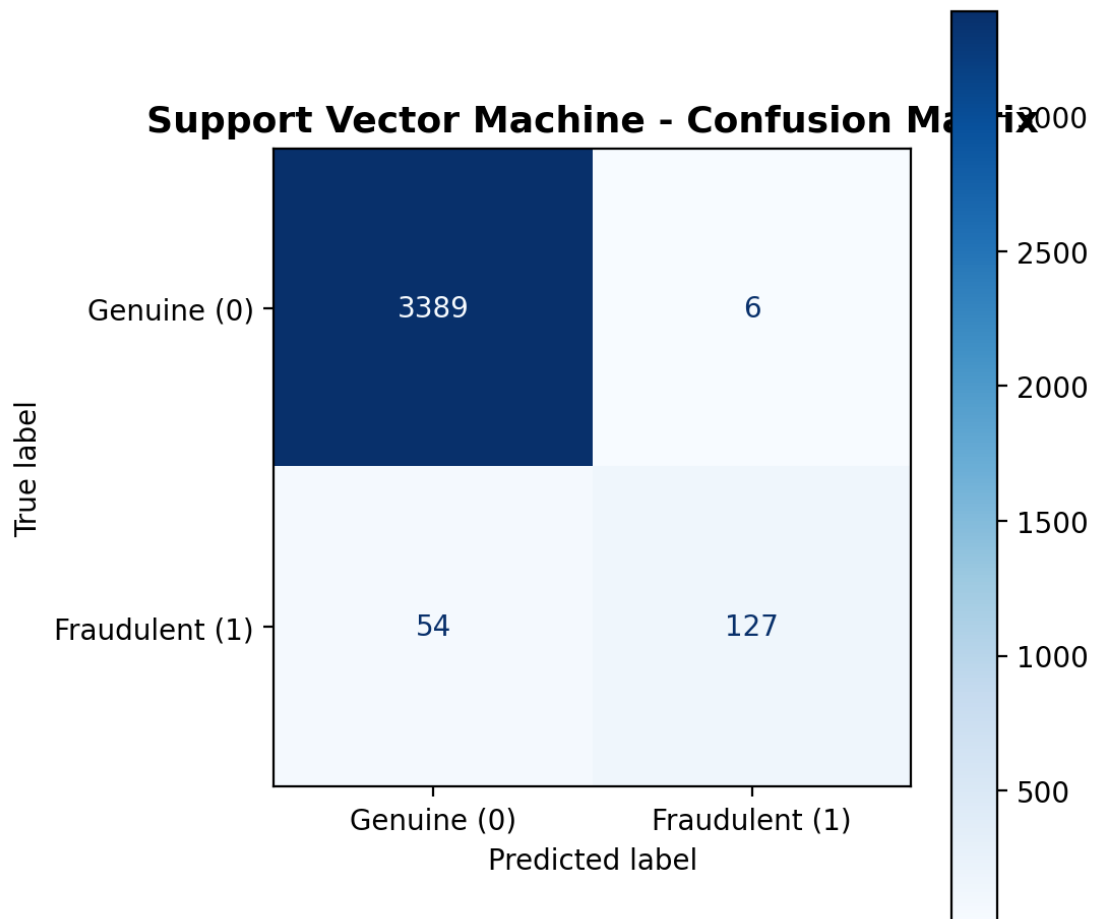
Advantages

- Excellent for text classification.
- Handles high-dimensional data efficiently.
- Produces high classification accuracy.

Limitations

- Training time increases for very large datasets.
- Hyperparameter tuning is important for achieving the best performance.

Figure 7: Support Vector Machine Confusion Matrix



Chapter 10: Hyperparameter Tuning

After evaluating the initial machine learning models, hyperparameter tuning was performed to further improve the performance of the Support Vector Machine.

Hyperparameter tuning helps identify the optimal configuration of model parameters that produces the best classification performance.

In this project, GridSearchCV from Scikit-learn was used to search different values of the regularization parameter (C) using 5-fold cross-validation.

The parameter grid included multiple values of C, and the model was evaluated using the F1-score during cross-validation.

The best hyperparameter combination obtained from GridSearchCV was then used to train the final Support Vector Machine model.

The tuned model showed improved fraud detection capability while maintaining high overall accuracy.

Chapter 11: Model Evaluation

To evaluate the performance of the implemented machine learning models, several evaluation metrics were used instead of relying only on accuracy.

The following metrics were calculated for every model:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Accuracy

Accuracy measures the percentage of correctly classified job postings.

Precision

Precision indicates how many job postings predicted as fraudulent were actually fraudulent.

Recall

Recall measures how many actual fraudulent job postings were successfully identified by the model.

F1-Score

F1-score provides a balanced measure by combining Precision and Recall into a single metric.

Confusion Matrix

The confusion matrix summarizes the classification results by showing:

- True Positives
- True Negatives

- False Positives
- False Negatives

These evaluation metrics provide a comprehensive understanding of model performance.

Chapter 12: Model Comparison

The performance of all machine learning algorithms was compared using the evaluation metrics obtained during testing.

Table 1: Model Comparison

Model	Accuracy
Support Vector Machine	0.983221
Tuned SVM	0.982383
Random Forest	0.979027
Logistic Regression	0.972315
Naive Bayes	0.962528

Based on the comparative analysis:

- Logistic Regression achieved good baseline performance.
- Naive Bayes performed efficiently but generated more false predictions.
- Random Forest improved classification performance but required more computational resources.
- Support Vector Machine consistently achieved the highest overall performance.
- Hyperparameter tuning further improved the reliability of the Support Vector Machine.

Therefore, the Support Vector Machine (Linear SVM) was selected as the final model for building the Fake Job Posting Detection System.

Chapter 13: Prediction System

After selecting the Support Vector Machine as the final model, a prediction system was developed to classify unseen job postings.

The prediction system follows the same preprocessing pipeline used during model training. When a new job posting is entered, the text is first cleaned by converting it to lowercase, removing special characters, numbers, extra spaces, and stopwords. Stemming is then applied to convert words into their root form.

The processed text is transformed into numerical features using the trained TF-IDF Vectorizer. Finally, the trained Support Vector Machine model predicts whether the job posting is genuine or fraudulent.

The system returns one of the following outputs:

- Genuine Job Posting
- Fraudulent Job Posting

The prediction system ensures that every new input is processed consistently, making it suitable for real-world applications where users need quick and reliable fraud detection.

Prediction Workflow

1. Input a new job posting.
2. Clean and preprocess the text.
3. Apply TF-IDF vectorization.
4. Use the trained SVM model for prediction.
5. Display the final result.

Figure 8: Prediction System Output

```

>>> new_job_post = """
... Congratulations!
...
... You have been selected without applying.
...
... Salary Rs.2,50,000 per month.
...
... No skills required.
...
... Pay Rs.1,999 security deposit today.
...
... Immediate joining.
...
... Only today offer.
...
... Contact on Telegram now.
... """
>>>
>>> prediction = tuned_svm.predict(job_vector)
>>> print(prediction)
[1]
>>> if prediction[0] == 1:
...     print("The job post is fraudulent.")
... else:
...     print("The job post is not fraudulent.")
The job post is fraudulent.

```


Chapter 14: Results and Discussion

The performance of all machine learning models was evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

The experimental results indicate that all models successfully classified genuine job postings with high accuracy. However, their ability to identify fraudulent job postings varied.

Logistic Regression provided a strong baseline model with good overall accuracy. Naive Bayes performed efficiently on text data and required very little computational time, although its fraud detection capability was comparatively lower.

Random Forest achieved competitive performance but required more training time because of its ensemble learning approach.

Among all the implemented algorithms, the Support Vector Machine consistently produced the best overall results. It achieved the highest classification accuracy and demonstrated superior performance in detecting fraudulent job postings.

After applying hyperparameter tuning using GridSearchCV, the Support Vector Machine achieved a better balance between precision and recall, making it the most suitable model for this project.

The comparative analysis clearly shows that Support Vector Machine is highly effective for Natural Language Processing tasks involving high-dimensional textual data.

Table 2: Final Model Performance

Model	Accuracy
Support Vector Machine	0.983221
Tuned SVM	0.982383
Random Forest	0.979027
Logistic Regression	0.972315
Naive Bayes	0.962528

Chapter 15: Conclusion

This project successfully developed a Machine Learning based Fake Job Posting Detection System capable of classifying job advertisements as genuine or fraudulent.

The project began with exploratory data analysis to understand the dataset and identify missing values. Text preprocessing techniques including lowercase conversion, removal of special characters, stopwords removal, and stemming were applied to improve text quality. The cleaned textual information was converted into numerical vectors using the TF-IDF Vectorizer.

Multiple machine learning algorithms including Logistic Regression, Multinomial Naive Bayes, Random Forest, and Support Vector Machine were trained and evaluated. Their performance was compared using multiple evaluation metrics instead of relying only on accuracy.

Experimental results demonstrated that the Support Vector Machine achieved the best overall performance. Hyperparameter tuning further improved the reliability of the model. Therefore, the tuned Support Vector Machine was selected as the final model for building the prediction system.

The developed system can automatically classify unseen job postings and has the potential to assist online recruitment platforms in reducing fraudulent job advertisements.

Overall, the project demonstrates how Natural Language Processing and Machine Learning techniques can be effectively applied to solve real-world recruitment fraud problems.

Chapter 16: Future Scope

Although the developed system performs well, several improvements can be made in future work.

Possible future enhancements include:

- Using advanced deep learning models such as LSTM, GRU, and BERT.
- Increasing the size and diversity of the training dataset.
- Applying Explainable Artificial Intelligence (XAI) techniques to understand model decisions.
- Developing a web application using Flask or Django.
- Deploying the model on cloud platforms for real-time prediction.
- Integrating the system with online recruitment websites.
- Performing continuous model retraining using newly collected job postings.

These improvements can further increase the accuracy, scalability, and practical usability of the Fake Job Posting Detection System.

Chapter 17: References

The following resources were used during the development of this project:

- [Scikit-learn Documentation](#).
- [NLTK Documentation](#).
- [Pandas Documentation](#).
- [NumPy Documentation](#).
- [Matplotlib Documentation](#).
- [Kaggle – Fake Job Postings Dataset](#).
- [Python Official Documentation](#).